DEFINICIÓN DEL PROYECTO

TuHospedaje

Definición del Proyecto

TuHospedaje — Plataforma de Reservas de Alojamientos Turísticos

ALCANCE

4 sprints y 35 historias oficiales

STACK

Spring Boot 4.0 · Java 17 · React 19 · MariaDB

FUERA DE ALCANCE

Pagos, mensajería interna y mapas

Equipo de Desarrollo · Mayo 2026

1. Identidad, Nicho y Propósito del Negocio

TuHospedaje es una plataforma web centralizada de reservas de alojamiento enfocada en **alojamientos turísticos**. El propósito principal del negocio es resolver la fricción entre los usuarios que buscan hospedaje de calidad y los administradores que necesitan gestionar su catálogo de manera eficiente. La plataforma permite a los clientes finales explorar un catálogo dinámico y detallado, mientras que provee a los administradores un panel de control intuitivo para crear, actualizar y dar de baja alojamientos.

- **Nombre Oficial:** TuHospedaje
- **Identidad Visual:** Isologotipo institucional para el tema claro vigente. El modo oscuro es trabajo futuro y no está soportado ni aprobado en la versión actual.
- **Ubicación del recurso:**
  `frontend/src/assets/images/TuHospedaje_Isologotipo.png`

2. Alcance del Proyecto y Hoja de Ruta (Roadmap)

El desarrollo se ejecutará de forma incremental a lo largo de **4 Sprints** planificados, garantizando un Producto Mínimo Viable (MVP) funcional desde el primer hito.

2.1. Matriz de Incremento por Sprint

SPRINT	FOCO ESTRATÉGICO	ENTREGABLES PRINCIPALES
Sprint 1	Base de la Solución	Catálogo interactivo de alojamientos, Panel de Administración (CRUD), identidad visual y maquetación responsiva.
Sprint 2	Seguridad y Organización	Autenticación/Registro con JWT, sistema de roles (Admin/User), categorización dinámica y gestión de características.
Sprint 3	Búsqueda y UX Avanzada	Buscador predictivo con filtros funcionales, sección de favoritos, módulo de políticas del alojamiento y sistema de puntuaciones/reseñas.
Sprint 4	Transaccionalidad	Motor de reservas (Booking), control de disponibilidad, historial de usuario, notificaciones por Email y canal de contacto vía WhatsApp.

2.2. Exclusiones Explícitas (Fuera de Alcance)

Para mitigar riesgos de desarrollo y asegurar la entrega en los tiempos estipulados, las siguientes características **no** forman parte de la solución actual:

- Procesamiento de pagos en línea (Pasarelas de pago externas).
- Chat de mensajería interna entre usuarios y administración.
- Mapas interactivos con geolocalización en tiempo real.

3. Arquitectura Técnica y Stack Tecnológico

La solución adopta una arquitectura desacoplada basada en el patrón de diseño **Client-Server**, estructurada en un monorrepositorio para facilitar el despliegue continuo.

```
[ Frontend: React 19 ]  ──( Peticiones HTTP / JSON )──>  [ Backend: Spring Boot 4.0 ]  ──>  [ Base de Datos: MariaDB ]
```

3.1. Infraestructura Tecnológica

CAPA	TECNOLOGÍA
Backend	Java 17 / Spring Boot 4.0 / Spring Data JPA / Flyway / Lombok
Frontend	React 19 / Vite / React Router (Single Page Application - SPA)
Base de Datos	MariaDB (Motor Relacional)
Almacenamiento	Local en backend (`/uploads/**`) con validación de magic bytes
Notificaciones	Outbox transaccional (`email_outbox`) y despacho asíncrono SMTP
Seguridad (Sprint 2+)	Spring Security + JSON Web Tokens (JWT)
Protocolo	API RESTful en `http://localhost:8080/api/`

3.2. Estándar de Diseño del Frontend

La interfaz vigente usa **CSS puro** mediante **Custom Properties** para el tema claro y los estados de la aplicación. La identidad visual, la paleta aprobada y las reglas de contraste están definidas por el Manual de Identidad Visual; `DESIGN.md` traduce esas reglas y registra el estado de implementación. El modo oscuro no tiene una paleta aprobada ni una ruta de activación y queda como trabajo futuro; este documento no define valores dark ni afirma soporte nativo.

4. Decisiones Arquitectónicas Clave (ADR)

4.1. Uso del Dominio Semántico `Lodging`

En lugar de utilizar términos genéricos como `Product`, todo el backend y frontend se alinean bajo el concepto de **Lodging** (Alojamiento). Esto asegura la coherencia del código con el modelo de negocio.

4.2. Modelo de Datos Extensible y Versionado

La propiedad `Lodging.category` se diseñó como una Foreign Key de tipo **nullable** en el Sprint 1 para permitir poblar la base de datos de manera inmediata. A partir de las etapas siguientes, el esquema de base de datos se versiona y evoluciona de forma controlada y reproducible mediante migraciones de

Flyway (`V1` a `V13`), garantizando integridad referencial, restricciones de unicidad y bloqueos de concurrencia optimista y pesimista.

4.3. Patrón de Backend Desacoplado

Se implementa un flujo estricto:

```
Controller → Service (Interface + Impl) → Repository → Entity / DTO
```

Los controladores se inyectan estrictamente a través de constructores (evitando la directiva `@Autowired` en atributos para facilitar pruebas unitarias), y la conversión de datos se centraliza en los DTOs mediante métodos estáticos de mapeo (`toEntity()` / `fromEntity()`).

4.4. Inyección de Configuración mediante Entornos

El Frontend se desacopla del entorno de ejecución utilizando variables de entorno (`.env`) mediante el prefijo `VITE_API_URL` , asegurando que la transición a servidores de producción sea transparente.

4.5. Bean Validation Declarativa

Se adopta la validación de entrada de datos mediante anotaciones declarativas JSR 380 en los DTOs y objetos Request. Las peticiones son interceptadas con `@Valid` a nivel de controlador de Spring Boot y cualquier error de validación es procesado de forma centralizada por el `GlobalExceptionHandler` capturando la excepción `MethodArgumentNotValidException` , permitiendo retornar respuestas HTTP 400 (Bad Request) estandarizadas y limpias para el cliente.

4.6. Dirección de iconografía

La dirección vigente es Lucide, de acuerdo con el Manual de Identidad Visual. Sus iconos vectoriales mantienen tamaños y etiquetas accesibles consistentes; el tree-shaking de Vite evita incluir iconos no utilizados.

El repositorio todavía conserva SVG inline, imágenes de Icons8, glifos de texto, estrellas y otros mecanismos heredados en superficies concretas. Esas excepciones son deuda de implementación registrada en `DESIGN.md` ; no constituyen una alternativa normativa equivalente ni cambian la dirección Lucide.

4.7. Tablas admin client-side (decisión histórica)

El alcance original de esta definición eligió el ordenamiento y la paginación client-side mediante `useTableData` para las tablas administrativas, incluida `AdminLodgings` . Esta decisión conserva el contexto histórico del proyecto.

4.8. Estado actual de las tablas administrativas

La implementación actual distingue dos modelos:

ÁREA	MODELO ACTUAL	ALCANCE
`AdminLodgings`	Server-driven	La búsqueda, el ordenamiento y la paginación se solicitan al backend.
`AdminReservations`	Server-driven	La consulta, los filtros, el ordenamiento y la paginación se solicitan al backend.
`AdminCategories`, `AdminFeatures`, `AdminPolicies`, `AdminUsers`	Client-side	La UI carga los registros y aplica el ordenamiento y la paginación localmente.

Los reportes históricos que describen todas las tablas administrativas como client-side corresponden a cortes anteriores y no describen por completo el estado actual.